```
for (int i=0; i<=brightness; i++) {
    analogWrite(backLight, i);
    delay(30);
  }
  for (int i=brightness; i>=0; i--) {
    analogWrite(backLight, i);
    delay(30);
  }
  lcd.clear();
```

So far, all of this code has happened and still nothing can be seen on the screen! To actually see the answer, we need to fade the backlight from 0 or off all the way up to the brightness level determined at the top of the sketch. Once we hit the brightness level, we fade it back down to off again and clear the LCD. We could just connect the LED pin to +5v instead, but where would the fun be in that? This little addition to the code creates much more intrigue and theatricality. Try it without it and I bet you won't like it as much.

With that rather convoluted explanation out of the way, let's back up and talk more specifically about how arrays work. We will also look at some different kinds of arrays and then discuss how these arrays affect the memory usage on the Arduino microcontroller. Hopefully after looking at the way the memory works, there will be reason enough for going back to figure out how we can utilize program memory better.

Arrays

At their simplest, arrays can turn a single variable into a list of multiple variables. Another way to look at it is that an array is a collection of common data elements chosen from any of the various data types available to variables. These elements are addressed by an index number that points to a specific data element. We used both numerical and character arrays in our project code to store our answers, pin assignments, and sensor readings. To put an array to use we begin with an array declaration.

Declaring Arrays

Declaring an array is very similar to declaring any other variable. The simplest form of array declaration can be seen in this following example:

```
int myArray[3];
```

This basic array creates three separate variables referred to in a sketch as `myArray[0]`, `myArray[1]`, and `myArray[2]`. Declaring an array in this manner tells the Arduino compiler to set aside a block of memory that we will fill with data at some point later on. To declare an array, we need at the very least three things: the data type of the array; the name for the array; and either a number surrounded by square brackets, `[ ]`, that is known as the **index**, or a list of values assigned to the array. The index tells the array how many individual variables, or **elements**, is a part of the array. The size of the array indicated by its index in the array declaration is collectively known as the array's **dimension**. In this case, this integer type array has a dimension of three variables that are each addressed individually by that element's index.

Arrays are 0 indexed, meaning that the first element begins at position 0 and the last element in the array has an index that is one less than the array's dimension size. That is why, in our last example in an array with three elements, the first element has an index of 0 and the last element is at index 2. Building